MODULE 24 · DOMAIN 6 - SECURITY AND COMPLIANCE

IAM at Scale

How a request is judged, roles and trust, confused deputies, permissions boundaries, session policies and ABAC, Identity Center, keyless workloads, Access Analyzer and root access.

DOMAIN 6 of 6 EXAM WEIGHT 17% MODULE 24 of 34

admin@sthithapragna.com

Every request is judged by a **stack of policies**. Some can grant, some can only cap, and one **explicit deny** anywhere wins.

The policy stack

POLICY TYPE	GRANTS OR CAPS?	ATTACHED TO
Identity-based policy	Grants	Users, groups, roles
Resource-based policy	Grants, including across accounts	Buckets, keys, queues, roles' trust
Permissions boundary	Caps identity-based grants	Users and roles
Session policy	Caps one session	Passed when assuming a role
SCP and RCP	Cap accounts and resources	Organisation, OUs, accounts

ON THE EXAM

Effective permissions are the overlap of every applicable grant and cap, minus any explicit deny.

Same account, or across accounts?

ACROSS ACCOUNTS

Both sides must allow

- Caller's identity policy must allow
- The resource's policy must allow the caller
- Either side alone is not enough
- Plus any organisation policies on each side

SAME ACCOUNT

Either side can allow

- An identity policy alone can be enough
- Or a resource policy alone
- Except KMS key policies and role trust policies
- Those must always allow explicitly

THE TIEBREAKER

Cross-account access denied with a correct resource policy: check the caller's own identity policy.

Roles and trust

- A role's **trust policy** says who may assume it; its **permissions policies** say what it can then do.
- Assuming a role returns temporary credentials from STS.
- Sessions last up to the role's maximum, as long as 12 hours; **role chaining** caps them at one hour.
- A trust policy naming a whole account trusts every principal there with permission to assume it.

> ON THE EXAM
>
> Cannot assume a role: check the trust policy first. Assumed, but cannot act: check the permissions policies.

The confused deputy

- A service acting for many customers could be tricked into using its access for the wrong one.
- For third parties: require an **sts:ExternalId** unique to you in the role's trust policy.
- For AWS services acting for you: condition on **aws:SourceArn** or **aws:SourceAccount**.
- The external ID is not a secret password. It stops one customer's configuration being reused against another.

> ON THE EXAM
>
> A SaaS vendor assumes roles in many customers' accounts: its trust policies need an external ID condition.

A trust policy for a third-party vendor

```
1 { "Version": "2012-10-17",
2   "Statement": [{
3     "Effect": "Allow",
4     "Principal": { "AWS": "arn:aws:iam::999988887777:role/VendorMonitoring" },
5     "Action": "sts:AssumeRole",
6     "Condition": {
7       "StringEquals": { "sts:ExternalId": "acme-7f3c9a21" } }
8   }] }
```

4 Trust one specific role in the vendor's account, not the whole account.

7 The vendor must present the ID it assigned to you.

ON THE EXAM

A vendor-facing role with no ExternalId condition is the confused deputy the exam wants you to spot.

Permissions boundaries

- A **boundary** sets the maximum permissions a user or role's identity policies can grant.
- The classic use: let developers create roles, but only roles that carry the boundary.
- Then no role a developer creates can exceed the boundary, so there is no privilege escalation.
- Also deny developers the ability to remove or change the boundary, or the protection is empty.

> ON THE EXAM
>
> Delegate role creation without allowing escalation: a permissions boundary enforced with iam:PermissionsBoundary.

Delegating role creation safely

```
1 { "Effect": "Allow",
2   "Action": ["iam:CreateRole", "iam:AttachRolePolicy", "iam:PutRolePolicy"],
3   "Resource": "arn:aws:iam::*:role/app-*",
4   "Condition": { "StringEquals": {
5     "iam:PermissionsBoundary": "arn:aws:iam::111122223333:policy/DevBoundary" } } },
6 { "Effect": "Deny",
7   "Action": ["iam:DeleteRolePermissionsBoundary", "iam:PutRolePermissionsBoundary"],
8   "Resource": "*" }
```

5 Roles can only be created with this boundary attached.

7 And the boundary can never be removed or swapped.

ON THE EXAM

Developers self-serve roles; the security team owns only the boundary.

Session policies and session tags

- A **session policy** passed when assuming a role narrows that session below the role's permissions.
- **Session tags** passed on assume or federation become aws:PrincipalTag values for that session.
- **Transitive** session tags survive role chaining.
- Useful when one broad role serves many callers, each needing a narrower view.

> ON THE EXAM
>
> One role, many callers, each limited differently: session policies or session tags, not one role per caller.

Role-based or attribute-based access?

ABAC

Match tags, not names

- One policy: principal's tag must equal resource's tag
- New projects need no new policies
- Tags come from users, roles or sessions
- Scales to hundreds of teams

RBAC

A role per job per project

- Policies list resources explicitly
- Every new project means new policies
- Clear and simple at small scale
- Explodes into hundreds of roles

THE TIEBREAKER

Hundreds of projects, same job functions, tagged resources: ABAC with aws:PrincipalTag and aws:ResourceTag.

IAM Identity Center

- Workforce sign-in to many accounts, with users and groups from your identity provider, synced by SCIM.
- **Permission sets** become roles in each account they are assigned to.
- **Attributes for access control** pass user attributes as session tags, enabling ABAC.
- Short-lived credentials for the console and CLI; no IAM users or long-lived keys.

> ON THE EXAM
>
> Workforce access to many accounts with corporate identities: Identity Center, not IAM users per account.

Workloads without long-lived keys

WORKLOAD	KEYLESS WAY TO GET AWS CREDENTIALS
EC2 instance	Instance profile role
ECS task, Lambda function	Task role, execution role
EKS pod	EKS Pod Identity, or IAM roles for service accounts
GitHub Actions and other CI systems	OIDC federation to an IAM role
On-premises servers	IAM Roles Anywhere, with X.509 certificates

ON THE EXAM

Access keys stored in a CI system's secrets are the answer to replace. OIDC federation replaces them.

IAM Access Analyzer

CAPABILITY	ANSWERS
External access findings	Which resources are shared outside my zone of trust?
Unused access findings	Which roles, keys and permissions are never used?
Policy validation	Is this policy valid and following best practice?
Custom policy checks	Does this change grant new access, or public access?
Policy generation	What does this role actually use, from CloudTrail?

ON THE EXAM

Set the zone of trust to the organisation, run it from a delegated administrator, send findings to Security Hub.

Custom policy checks in a pipeline

- **check-no-new-access**: does the new policy grant anything the reference policy did not?
- **check-access-not-granted**: does it allow specific sensitive actions, such as iam:PassRole on everything?
- **check-no-public-access**: would a resource policy make the resource public?
- Run them in CodeBuild on IaC changes; a failing check fails the build.

> ON THE EXAM
>
> Block risky permission changes before deployment, automatically: custom policy checks in the pipeline.

Centralised root access management

- Since November 2024, root credentials can be **removed** from member accounts entirely.
- New member accounts can be created with no root credentials at all.
- Root-only tasks run as short **privileged root sessions** from the management or a delegated admin account.
- For example: unlocking an S3 bucket policy that denies everyone, or an SQS queue policy.

> ON THE EXAM
>
> No root passwords, MFA devices or keys to manage in member accounts, yet root tasks still possible: centralised root access.

> Developers deploy their own Lambda functions and must create an IAM execution role for each one. The central security team is a bottleneck for role requests.
>
> The company must let developers create roles without being able to grant themselves or their functions more permissions than an approved maximum.

What should the DevOps engineer implement?

A. Allow iam:CreateRole only with the approved permissions boundary attached, and deny removing or changing boundaries

B. Keep all role creation with the central security team

C. Give developers AdministratorAccess and an SCP that denies iam:*

D. Let developers create any role, and review IAM Access Analyzer findings weekly

◆ PAUSE AND DECIDE

What should the DevOps engineer implement?

A Boundary-enforced role creation

Developers self-serve, and no role they create can exceed the boundary.

B Central team keeps it

Keeps the bottleneck the company wants removed.

C Admin plus SCP denying IAM

Blocks all role creation, which developers need.

D Anything, reviewed weekly

Detects escalation a week after it happens. Not prevention.

THE SIGNAL

Delegate IAM safely: permissions boundaries enforced by condition, and protected from removal.

> A company lets a SaaS monitoring vendor read its CloudWatch data by assuming a role in the company's account. The vendor serves thousands of customers from one AWS account.
>
> The security team worries that another vendor customer could trick the vendor into reading this company's data.

What should the DevOps engineer add to the role's trust policy?

A Replace the role with an IAM user and access keys for the vendor

B A condition requiring the sts:ExternalId value the vendor assigned to this company

C A condition limiting access to the vendor's IP address range

D A condition requiring MFA

◆ PAUSE AND DECIDE

What should the DevOps engineer add to the role's trust policy?

A **IAM user and keys**

Long-lived credentials, a worse risk, and it does not address the deputy problem.

B **sts:ExternalId condition**

The vendor must present this company's ID, so another customer cannot redirect it here.

C **IP condition**

Every customer's requests come from the same vendor IPs. It cannot tell them apart.

D **MFA**

An automated vendor system cannot provide MFA, and it still would not tell customers apart.

THE SIGNAL

Third party, many customers, one vendor account: the confused deputy. The answer is an external ID.

> A company has 300 projects, each with developers and data engineers. All resources carry a project tag. Staff sign in through IAM Identity Center, and the directory records each person's project.
>
> Creating roles and policies for every project has become unmanageable. New projects start every week.

Which approach scales with the LEAST operational overhead?

A One permission set per project and job function

B One IAM group per project in every account

C Pass project as a session tag with attributes for access control, and use one permission set per job with aws:ResourceTag/project matching aws:PrincipalTag/project

D Resource-based policies naming each project's users on every resource

◆ PAUSE AND DECIDE

Which approach scales with the LEAST operational overhead?

A Permission set per project

600 permission sets, and more every week. The problem, repeated.

B Groups per project

Still per-project objects in every account.

C ABAC through Identity Center

Two permission sets total; new projects need only tags, not new policies.

D Resource policies per resource

Thousands of policies to keep in step with staff changes.

THE SIGNAL

Many projects, same jobs, tagged resources, attributes in the directory: ABAC, not more roles.

> A deployment pipeline's role has broad permissions granted years ago. The security team wants it reduced to only the actions the pipeline has actually used in the last 90 days.
>
> The team wants the least manual effort.

What should the DevOps engineer use?

A Read 90 days of CloudTrail logs by hand and write a new policy

B AWS Trusted Advisor security checks

C Remove permissions one at a time until the pipeline breaks

D IAM Access Analyzer policy generation from the role's CloudTrail activity

◆ PAUSE AND DECIDE

What should the DevOps engineer use?

A Read CloudTrail by hand

Accurate but enormously laborious. Policy generation does it for you.

B Trusted Advisor

Broad account checks. It does not build a least-privilege policy for a role.

C Remove until broken

Causes outages, and still leaves guesswork.

D Access Analyzer policy generation

Builds a policy from the role's actual CloudTrail activity, ready to review and apply.

THE SIGNAL

'Only what it actually used': policy generation from CloudTrail. Unused access findings help spot the excess.

> Teams submit CloudFormation changes that include IAM policies. The security team wants the pipeline to fail automatically if a change grants permissions beyond an approved reference policy, or makes any resource public.
>
> It wants this before deployment, with no manual review.

What should the DevOps engineer add to the pipeline?

A. A CodeBuild step running IAM Access Analyzer custom policy checks: check-no-new-access and check-no-public-access

B. A manual approval where the security team reads each policy

C. An AWS Config rule that evaluates IAM policies after deployment

D. Amazon GuardDuty in the target accounts

◆ PAUSE AND DECIDE

What should the DevOps engineer add to the pipeline?

A Custom policy checks in CodeBuild

Each check answers one requirement, automatically, before anything deploys.

B Manual approval

Ruled out: no manual review.

C Config after deployment

Too late: the permissions already exist.

D GuardDuty

Detects threats in running accounts. It does not review policy changes.

THE SIGNAL

Beyond a reference: check-no-new-access. Public: check-no-public-access. Both in the build, before deploy.

> An organisation has 120 member accounts, each with a root password and MFA device held in a safe. Auditors want no root credentials to exist in member accounts.
>
> The security team must still be able to perform root-only tasks, such as unlocking a bucket policy that denies everyone.

What should the DevOps engineer do?

A. Rotate every root password and move the MFA devices to a second safe

B. Enable centralised root access management, remove member root credentials, and use privileged root sessions when needed

C. Attach an SCP that denies all actions by the root user

D. Close the member accounts and recreate them

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Rotate and re-store

Credentials still exist, 120 times over.

B Centralised root access

Root credentials removed; root-only tasks done as short, scoped privileged sessions.

C SCP denying root

Credentials still exist, and root-only tasks become impossible.

D Recreate accounts

Enormous disruption, and new accounts would still need the same feature.

THE SIGNAL

No root credentials, yet root tasks still possible: centralised root access with privileged sessions.

Module 24 in five lines

- Policies grant or cap; an explicit deny wins. Cross-account needs both sides to allow.
- Trust says who; permissions say what. External IDs stop confused deputies.
- Boundaries make delegation safe; session tags and ABAC make permissions scale.
- Identity Center for people; roles, OIDC and Roles Anywhere for workloads. No long-lived keys.
- Access Analyzer: find, trim, and check in the pipeline. Remove member root credentials.

> ON THE EXAM
>
> Next, module 25: secrets, keys and certificates.